Modül 28

Authentication & Authorization

Login, JWT, Roles, Permissions

Modül:	28 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

🚀 Modül 28'e Hoş Geldin

Kimlik doğrulama (authentication) ve yetkilendirme (authorization) — her enterprise uygulamanın temeli. TSE'de farklı rol ve yetkilerle çalışan uygulamalar yazıyorsun. Bu modülde production-grade auth sistemi öğreneceğiz.

Öğrenecekler:

- ✓ Authentication vs Authorization farkı
- ✓ JWT-based auth flow
- ✓ Token storage strategies
- ✓ Refresh token pattern
- ✓ Auth interceptor
- ✓ Route guards (auth, role, permission)
- ✓ Permission-based UI rendering

DERS 28.1

Auth Kavramları

Authentication, Authorization, Identity.

🔐 Authentication (Kimlik Doğrulama)

Kimsin sen? Kullanıcının kim olduğunu doğrulama.

- Username + Password
- OAuth (Google, GitHub)
- SSO (Single Sign-On)
- 2FA (Two-Factor Authentication)

🔑 Authorization (Yetkilendirme)

Ne yapabilirsin? Authenticated kullanıcının hangi işlemleri yapabileceği.

- Role-based (RBAC) — admin, editor, viewer
- Permission-based — read:users, write:posts, delete:any
- Attribute-based (ABAC) — kullanıcı + kaynak + context
- Hierarchical — admin > manager > user

⚖️ Modeller

Model	Örnek	Use Case
RBAC	role: "admin"	Basit sistemler
Permission	permissions: ["read:users"]	Granular control
ABAC	user.dept === doc.dept	Complex policies
Resource-based	owner === currentUser	CRUD apps

DERS 28.2

JWT-Based Auth Flow

Token-based authentication.

Flow

```
1. Client: POST /api/auth/login { email, password }
2. Server: Validate credentials
3. Server: Generate JWT (signed with secret)
4. Server: Return { token, refreshToken, user }
5. Client: Store token
6. Client: Add to Authorization header (Bearer token)
7. Server: Validate token on each request
8. Server: Extract user from token claims
9. Token expires (15 min)
10. Client: Refresh with refreshToken
11. Server: Return new token
```

JWT Yapısı

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjMiLCJlbWFpbCI6ImpAZXhhbXBsZS5jb20iLCJleHAiOjE3MDAwMDAwMDB9.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

3 parça (. ile ayrılmış):
1. Header  : { "alg": "HS256", "typ": "JWT" }
2. Payload : { "sub": "123", "email": "j@example.com", "exp": 1700000000 }
3. Signature: HMACSHA256(base64(header)+"."+base64(payload), secret)
```

⚠ JWT Best Practices

- ✓ Kısa expire — 15 dakika ideal
- ✓ Refresh token ayrı, uzun süreli, single-use
- ✓ Signing algorithm — HS256 veya RS256
- ✓ Secret key uzun ve gizli (32+ char)
- ✓ Sensitive data payload'a koyma (base64 görülebilir)
- ✓ HTTPS zorunlu (man-in-the-middle önleme)

DERS 28.3

Auth Service Implementation

Production-grade auth service.

AuthService — Yapı

```
interface AuthState {
  user: User | null;
  token: string | null;
  refreshToken: string | null;
}

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);

  // State
  private _user = signal<User | null>(this.loadUser());
  private _token = signal<string | null>(this.loadToken());

  // Public readonly
  readonly user = this._user.asReadonly();
  readonly isLoggedIn = computed(() => this._token() !== null);
  readonly userRole = computed(() => this._user()?.role);

  constructor() {
    // Token expire watcher
    effect(() => {
      const token = this._token();
      if (token && this.isTokenExpired(token)) {
        this.refreshAccessToken();
      }
    });
  }

  // Methods continued...
}
```

Login Method

```
async login(email: string, password: string): Promise<void> {
  try {
    const response = await firstValueFrom(
      this.http.post<LoginResponse>('/api/auth/login', { email, password })
    );

    this.setSession(response);

    // Redirect to intended page or dashboard
    const returnUrl = this.route.snapshot.queryParams['returnUrl'] || '/dashboard';
    await this.router.navigateByUrl(returnUrl);
  } catch (error: any) {
    if (error.status === 401) {
      throw new Error('Email veya şifre hatalı');
    }
    throw new Error('Giriş başarısız');
  }
}

private setSession(response: LoginResponse) {
  localStorage.setItem('token', response.token);
  localStorage.setItem('refreshToken', response.refreshToken);
  localStorage.setItem('user', JSON.stringify(response.user));

  this._token.set(response.token);
  this._user.set(response.user);
}
```

Logout

```
async logout(): Promise<void> {
  try {
    // Backend'e logout bildir (refresh token'ı invalidate et)
    await firstValueFrom(
      this.http.post('/api/auth/logout', {
        refreshToken: this.getRefreshToken()
      })
    );
  } catch {
    // Logout backend'de başarısız olsa bile client'ta temizle
  }

  this.clearSession();
  this.router.navigate(['/login']);
}

private clearSession() {
  localStorage.removeItem('token');
  localStorage.removeItem('refreshToken');
  localStorage.removeItem('user');

  this._token.set(null);
  this._user.set(null);
}
```

DERS 28.4

Refresh Token Pattern

Token expire olduğunda otomatik yenile.

Refresh Logic

```
@Injectable({...})
export class AuthService {
  private refreshing = false;
  private refreshSubject = new BehaviorSubject<string | null>(null);

  async refreshAccessToken(): Promise<string> {
    // Concurrent requests için tek refresh
    if (this.refreshing) {
      return firstValueFrom(
        this.refreshSubject.pipe(filter(token => token !== null))
      ) as Promise<string>;
    }

    this.refreshing = true;
    this.refreshSubject.next(null);

    try {
      const refreshToken = this.getRefreshToken();
      if (!refreshToken) throw new Error('No refresh token');

      const response = await firstValueFrom(
        this.http.post<LoginResponse>('/api/auth/refresh', { refreshToken })
      );

      this.setSession(response);
      this.refreshSubject.next(response.token);

      return response.token;
    } catch (err) {
      this.clearSession();
      this.router.navigate(['/login']);
      throw err;
    } finally {
      this.refreshing = false;
    }
  }

  isTokenExpired(token: string): boolean {
    try {
      const decoded = jwtDecode<{ exp: number }>(token);
      return Date.now() >= (decoded.exp - 60) * 1000;  // 60s buffer
    } catch {
      return true;
    }
  }
}
```

DERS 28.5

Auth Interceptor

Her HTTP isteğine token ekle, 401 olunca refresh.

Interceptor

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { catchError, switchMap, throwError } from 'rxjs';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);

  // Skip auth endpoint'leri için
  if (req.context.get(SKIP_AUTH)) {
    return next(req);
  }

  const token = auth.getToken();
  if (token) {
    req = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
  }

  return next(req).pipe(
    catchError(error => {
      if (error instanceof HttpErrorResponse && error.status === 401) {
        return handleUnauthorized(req, next, auth);
      }
      return throwError(() => error);
    })
  );
};

function handleUnauthorized(req: HttpRequest<any>, next: HttpHandlerFn, auth:
AuthService) {
  return from(auth.refreshAccessToken()).pipe(
    switchMap(newToken => {
      const updatedReq = req.clone({
        setHeaders: { Authorization: `Bearer ${newToken}` }
      });
      return next(updatedReq);
    }),
    catchError(refreshError => {
      auth.logout();
      return throwError(() => refreshError);
    })
  );
}
```

⚙ Setup

```
// app.config.ts
import { provideHttpClient, withInterceptors } from '@angular/common/http';

providers: [
  provideHttpClient(
    withInterceptors([authInterceptor])
  )
]
```

DERS 28.6

Route Guards

Yetkisiz route erişimlerini engelle.

Auth Guard

```
import { CanActivateFn } from '@angular/router';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};
```

Role Guard (Factory)

```
export const roleGuard = (allowedRoles: string[]): CanActivateFn => {
  return (route, state) => {
    const auth = inject(AuthService);
    const router = inject(Router);
    const toast = inject(MessageService);

    if (!auth.isLoggedIn()) {
      return router.createUrlTree(['/login'], {
        queryParams: { returnUrl: state.url }
      });
    }

    const role = auth.userRole();
    if (role && allowedRoles.includes(role)) {
      return true;
    }

    toast.add({
      severity: 'error',
      summary: 'Yetkisiz',
      detail: 'Bu sayfaya erişim yetkiniz yok'
    });
    return router.createUrlTree(['/']);
  };
};

// Kullanım
{
  path: 'admin',
  canActivate: [authGuard, roleGuard(['admin'])],
  loadChildren: () => import('./admin/admin.routes').then(m => m.routes)
}
```

🔐 Permission Guard

```
export const permissionGuard = (requiredPermissions: string[]): CanActivateFn => {
  return (route, state) => {
    const auth = inject(AuthService);
    const router = inject(Router);

    if (!auth.isLoggedIn()) {
      return router.createUrlTree(['/login']);
    }

    const userPermissions = auth.user()?.permissions || [];
    const hasAll = requiredPermissions.every(p => userPermissions.includes(p));

    return hasAll || router.createUrlTree(['/forbidden']);
  };
};

// Kullanım
{
  path: 'users/manage',
  canActivate: [permissionGuard(['users:write', 'users:delete'])],
  component: UserManagementComponent
}
```

DERS 28.7

Permission-Based UI

Yetkiye göre UI elemanlarını göster/gizle.

hasPermission Directive

```
@Directive({
  selector: '[hasPermission]',
  standalone: true,
})
export class HasPermissionDirective {
  private templateRef = inject(TemplateRef);
  private viewContainer = inject(ViewContainerRef);
  private auth = inject(AuthService);

  hasPermission = input.required<string | string[]>();

  constructor() {
    effect(() => {
      const required = this.hasPermission();
      const required$ = Array.isArray(required) ? required : [required];

      const userPermissions = this.auth.user()?.permissions || [];
      const hasAccess = required$.every(p => userPermissions.includes(p));

      this.viewContainer.clear();
      if (hasAccess) {
        this.viewContainer.createEmbeddedView(this.templateRef);
      }
    });
  }
}
```

Template Kullanımı

```
<!-- Tek permission -->
<button *hasPermission="'users:delete'" (click)="delete()">Sil</button>

<!-- Birden fazla permission (hepsi gerekli) -->
<div *hasPermission="['admin:full', 'users:write']">
  Admin paneli
</div>

<!-- @if alternatifi (modern) -->
@if (auth.hasPermission('users:delete')) {
  <button (click)="delete()">Sil</button>
}
```

AuthService'e Permission Helper

```
@Injectable({...})
export class AuthService {
  // ... existing code

  hasPermission(permission: string): boolean {
    return this.user()?.permissions?.includes(permission) ?? false;
  }

  hasAnyPermission(permissions: string[]): boolean {
    const userPerms = this.user()?.permissions || [];
    return permissions.some(p => userPerms.includes(p));
  }

  hasAllPermissions(permissions: string[]): boolean {
    const userPerms = this.user()?.permissions || [];
    return permissions.every(p => userPerms.includes(p));
  }

  hasRole(role: string): boolean {
    return this.userRole() === role;
  }

  hasAnyRole(roles: string[]): boolean {
    const userRole = this.userRole();
    return userRole !== null && roles.includes(userRole);
  }
}
```

Computed Permissions

```
@Component({...})
export class ProjectComponent {
  private auth = inject(AuthService);

  canEdit = computed(() =>
    this.auth.hasPermission('projects:write') &&
    this.project()?.ownerId === this.auth.user()?.id
  );

  canDelete = computed(() =>
    this.auth.hasRole('admin') ||
    this.project()?.ownerId === this.auth.user()?.id
  );

  // Template'te
  // @if (canEdit()) { ... }
}
```

Modül 28 Özeti

Auth artık tam donanımındasın. Login, JWT, refresh, role/permission-based access — production-grade auth sistemi kurabilirsin.

Neler Öğrendin?

- ✓ Authentication vs Authorization farkı
- ✓ JWT-based auth flow
- ✓ AuthService implementation
- ✓ Refresh token pattern (concurrent-safe)
- ✓ Auth interceptor (auto-refresh on 401)
- ✓ Route guards (auth, role, permission)
- ✓ Permission-based UI rendering

Sıradaki Modül

Modül 29 — Advanced TypeScript Patterns. Type-level programming, generics, utility types, conditional types.